

BUỔI 4

KẾT NỐI SQL SERVER & SEQUELIZE ORM

Nâng cấp Backend Quản lý Sinh viên từ dữ liệu giả → dữ liệu thật

NODE.JS | TYPESCRIPT | SEQUELIZE 6 | SQL SERVER

TRƯỚC	Service → mảng trong RAM
HÔM NAY	Service → Model → Sequelize → SQL Server

Mục tiêu cuối: thêm SV003 bằng Postman → restart Node.js → SV003 vẫn còn.

LỘ TRÌNH BUỔI 4

Mỗi khái niệm lý thuyết đi kèm một bài thực hành kiểm chứng ngay trên project đang có

1. ÔN LẠI | 2. DATABASE | 3. ORM | 4. KẾT NỐI | 5. MODEL & SERVICE | 6. TEST & REVIEW

LỘ TRÌNH TỪ RAM ĐẾN DATABASE

1. **Ôn lại** — API đang đọc mảng RAM
2. **Database** — Thiết kế bảng + PK/FK
3. **ORM** — Hiểu Object ↔ Table
4. **Kết nối** — .env → database.ts → authenticate
5. **Model & Service** — findAll / findByPk / create
6. **Test & Review** — Postman + SSMS + restart

BẰNG CHỨNG THÀNH CÔNG — **POST 201 → SELECT thấy dữ liệu → restart server → GET vẫn đọc được cùng bản ghi**

THỰC HÀNH

Bài chính xuyên suốt

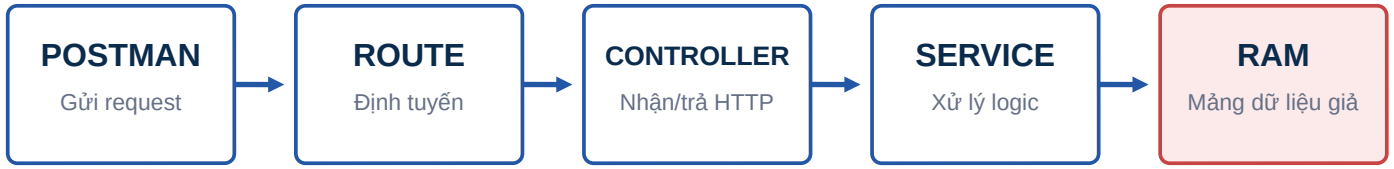
1. Không tạo project mới. Tiếp tục backend-quan-ly-sinh-vien.
2. Giữ URL, DTO, response success/message/data và các validation đã học.
3. Thay nguồn dữ liệu: RAM → SQL Server qua Sequelize.
4. Nộp bằng chứng Postman + SSMS + restart.

ĐIỂM DỪNG — API đọc/ghi SQL Server thật, không còn mảng giả trong đường đi chính.

ÔN LẠI PROJECT BUỔI TRƯỚC

Tìm đúng “điểm yếu” của hệ thống hiện tại trước khi thêm Database

LÝ THUYẾT - DỮ LIỆU ĐANG NẰM Ở ĐÂU?



VẤN ĐỀ

Tắt server hoặc khởi động lại → mảng RAM quay về trạng thái ban đầu. Đây chưa phải nơi lưu dữ liệu thật.

Route
Nhận đường dẫn + HTTP method, chuyển tới handler.

Controller
Nhận Request, gọi Service, trả Response.

Service
Chứa logic nghiệp vụ, không xử lý HTTP trực tiếp.

RAM
Bộ nhớ tạm của tiến trình; restart là mất trạng thái.

THỰC HÀNH

Checkpoint 0 - giữ nền đang chạy

1. Mở đúng project cũ; không pnpm init lại.
2. Chạy node -v, pnpm -v, pnpm build, pnpm dev.
3. Test GET /, GET /api/v1/sinh-vien, GET theo mã.
4. Commit/backup phiên bản đang chạy trước khi sửa.

PROMPT GỢI Ý

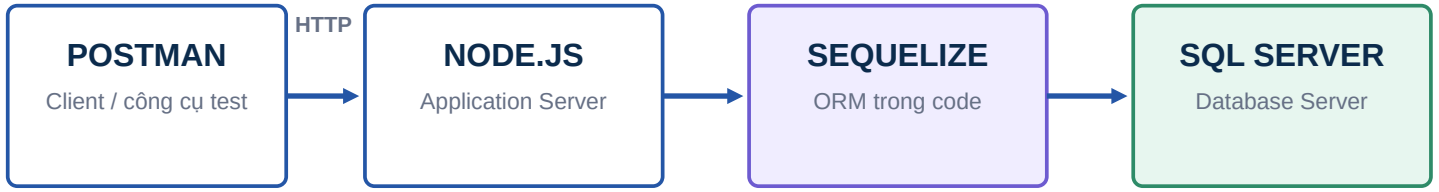
Yêu cầu AI đọc file hiện tại và liệt kê file cần thêm/sửa; chưa viết code.

ĐIỂM DỪNG Project cũ build và API cũ hoạt động ổn định trước khi chuyển Database.

DATABASE NẰM Ở ĐÂU TRONG BACKEND?

Phân biệt Application Server, ORM và Database Server

LÝ THUYẾT - KIẾN TRÚC KẾT NỐI



GHI NHỚ **Node.js xử lý nghiệp vụ. SQL Server mới là nơi lưu dữ liệu. Sequelize chỉ là "cầu nối" giúp code thao tác Database thuận tiện hơn.**

Application Server = nơi chạy logic

Database Server = nơi lưu dữ liệu

ORM = lớp ánh xạ trong code

THỰC HÀNH

Chuẩn bị SQL Server local

1. Xác nhận Database Engine đang chạy; SSMS chỉ là công cụ quản lý.
2. Bật TCP/IP, cấu hình port (ví dụ 1433) và restart Engine.
3. Kết nối SSMS theo tcp:127.0.0.1,<port>.
4. Bật SQL Server + Windows Authentication nếu dùng SQL login.

ĐIỂM DỪNG SSMS kết nối được đúng Engine qua TCP trước khi cấu hình Sequelize.

THIẾT KẾ BẢNG TRƯỚC KHI VIẾT MODEL

Database schema phải rõ trước khi yêu cầu AI sinh code

LÝ THUYẾT - TABLE / COLUMN / PK / FK

dbo.SinhVien

maSV	VARCHAR(5)	PK · NOT NULL
hoTen	NVARCHAR(150)	NOT NULL
email	NVARCHAR(254)	NOT NULL
maLop	NVARCHAR(50)	NOT NULL

Table
Một nhóm bản ghi cùng loại.

Column
Một thuộc tính của bản ghi.

Primary Key
Định danh duy nhất mỗi bản ghi.

Foreign Key
Liên kết bản ghi sang bảng khác.

Vì sao maSV là PK?

Project đã nhận diện sinh viên bằng maSV. Bài chính chưa cần thêm id tự tăng.

THỰC HÀNH

Tạo Database + bảng + seed

1. Tạo sql/01-quan-ly-sinh-vien.sql: Database + dbo.SinhVien.
2. Tạo sql/02-seed.sql: SV001, SV002; chạy lặp không trùng.
3. Kiểm tra Columns và PK trong SSMS.
4. SELECT dữ liệu và chụp bằng chứng.

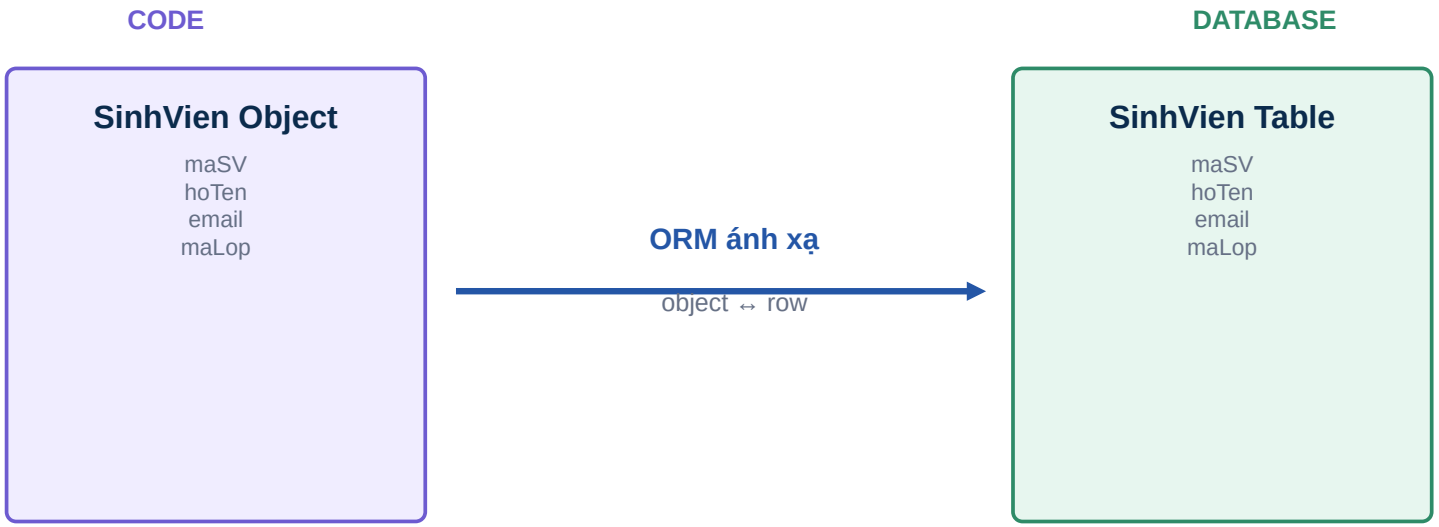
PROMPT GỢI Ý
Mô tả schema chính xác, yêu cầu AI tạo script SQL; cấm tự thêm id/timestamps/UNIQUE email.

ĐIỂM DỪNG Có đúng SV001, SV002; Unicode tiếng Việt hiển thị đúng; không DROP dữ liệu.

ORM LÀ GÌ?

Hiểu khái niệm trước khi học Sequelize

LÝ THUYẾT - OBJECT RELATIONAL MAPPING



SQL THUẦN

```
SELECT * FROM SinhVien;
```

ORM

```
SinhVien.findAll();
```

THỰC HÀNH

Nhận diện ORM trong code

1. So sánh một câu SELECT với findAll().
2. Giải thích ORM không thay thế Database và không làm SQL “biến mất”.
3. Phân biệt DTO (dữ liệu trao đổi) với Model (ánh xạ bảng).

ĐIỂM DỪNG Sinh viên nói được “SQL Server lưu dữ liệu; ORM chỉ giúp code thao tác dữ liệu”.

SEQUELIZE NẰM Ở ĐÂU TRONG STACK?

Runtime, ORM, driver và Database là bốn vai trò khác nhau

LÝ THUYẾT - STACK CỦA BUỔI HỌC

NODE.JS — Runtime chạy Backend

↓

SEQUELIZE 6 — ORM ánh xạ Model ↔ Table

↓

TEDIOUS — Driver giao tiếp SQL Server

↓

SQL SERVER — Database lưu bản ghi

Câu sai cần tránh: “Dữ liệu lưu trong Sequelize.”

THỰC HÀNH

Cài thư viện + tài khoản ứng dụng

1. Cấp SQL login sv_app chỉ SELECT + INSERT cho dbo.SinhVien.
2. Chạy pnpm add sequelize@6 tedious.
3. Giữ pnpm/ESM/NodeNext; không đổi sang npm/CommonJS.
4. Không dùng sa cho ứng dụng.

ĐIỂM DỪNG package.json giữ nền cũ; app login đủ quyền GET + POST nhưng không thừa quyền.

NÂNG CẤP CẤU TRÚC PROJECT

Không viết lại hệ thống - chỉ bổ sung đúng tầng dữ liệu cần thiết

LÝ THUYẾT - FILE NÀO MỚI? FILE NÀO GIỮ?

```
backend-quan-ly-sinh-vien/
├─ sql/
│  ├─ 01-quan-ly-sinh-vien.sql ★
│  ├─ 02-seed.sql              ★
│  └─ 03-app-login.sql         ★
└─ src/
   ├─ config/database.ts       ★
   ├─ models/SinhVien.ts       ★
   ├─ scripts/check-db.ts      ★
   ├─ services/sinhVien.service.ts ~
   ├─ controllers/sinhVien.controller.ts ~
   ├─ routes/sinhVien.routes.ts =
   └─ server.ts                ~
```

★ thêm mới | ~ sửa | = giữ

config/
Cấu hình kết nối Database.

models/
Ánh xạ cấu trúc bảng sang code.

scripts/
Lệnh nhỏ kiểm tra DB độc lập với HTTP server.

ESM lưu ý File .ts nhưng import nội bộ dùng đuôi .js.

THỰC HÀNH

Tạo đúng cây thư mục

1. Tạo sql/, src/config/, src/models/, src/scripts/ nếu chưa có.
2. Không sửa dist; không tạo app.ts mới chỉ để đổi kiến trúc.
3. Giữ Route, DTO, middleware và URL cũ.
4. Kiểm tra import nội bộ dùng .js với NodeNext.

ĐIỂM DỪNG Sinh viên chỉ ra được mỗi file mới nằm ở đâu và chịu trách nhiệm gì.

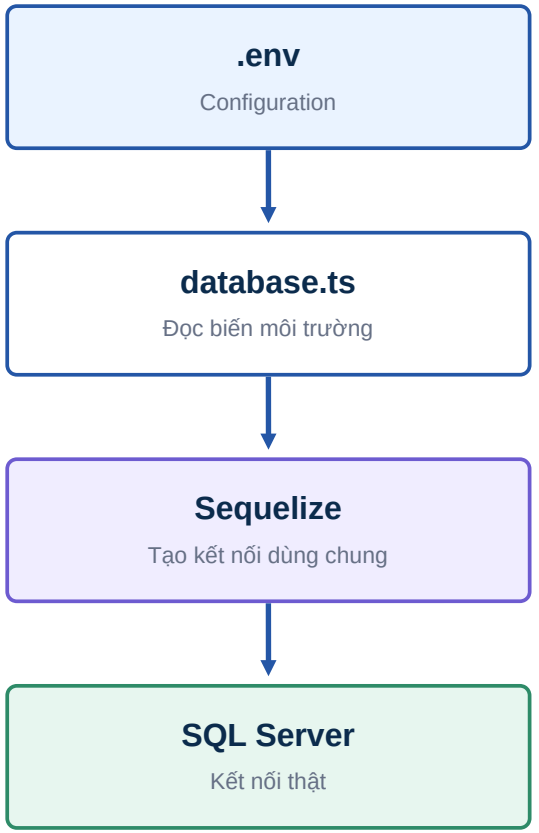
ENVIRONMENT VARIABLES & .ENV

Tách cấu hình khỏi source code và tránh hard-code thông tin nhạy cảm

LÝ THUYẾT - CODE ≠ CONFIGURATION

```env
.env

PORT=5000
DB_HOST=127.0.0.1
DB_PORT=1433
DB_NAME=QuanLySinhVienDB
DB_USER=sv_app
DB_PASSWORD=...
DB_ENCRYPT=true
DB_TRUST_CERT=true
```



THỰC HÀNH

Bổ sung .env và .env.example

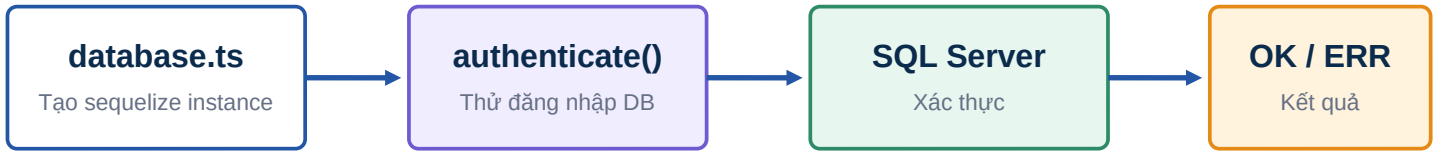
1. Đặt .env cùng cấp package.json, không đặt trong src.
2. Mật khẩu thật chỉ nằm trong .env local; .env.example dùng placeholder.
3. Phân biệt PORT của Express với DB_PORT của SQL Server.
4. Giữ .env trong .gitignore.

ĐIỂM DỪNG Không hard-code password; .env.example có thể nộp/Git mà không lộ secret.

KẾT NỐI DATABASE: AUTHENTICATE + TRY/CATCH

Tách bước “kết nối được” khỏi bước “Model đọc đúng bảng”

LÝ THUYẾT - CHECK KẾT NỐI ĐỘC LẬP




```
try    await sequelize.authenticate();      → kết nối thành công

catch  (error)                              → ghi lỗi chẩn đoán, không lộ mật khẩu
```

Script check-db dùng xong thì close(); server thật thì giữ kết nối mở.

THỰC HÀNH

Tạo database.ts + check-db.ts

1. Tạo src/config/database.ts, dialect="mssql" và đọc biến .env.
2. Tạo src/scripts/check-db.ts gọi sequelize.authenticate().
3. Thêm script check:db vào package.json.
4. Chạy pnpm build → pnpm check:db.

PROMPT GỢI Ý

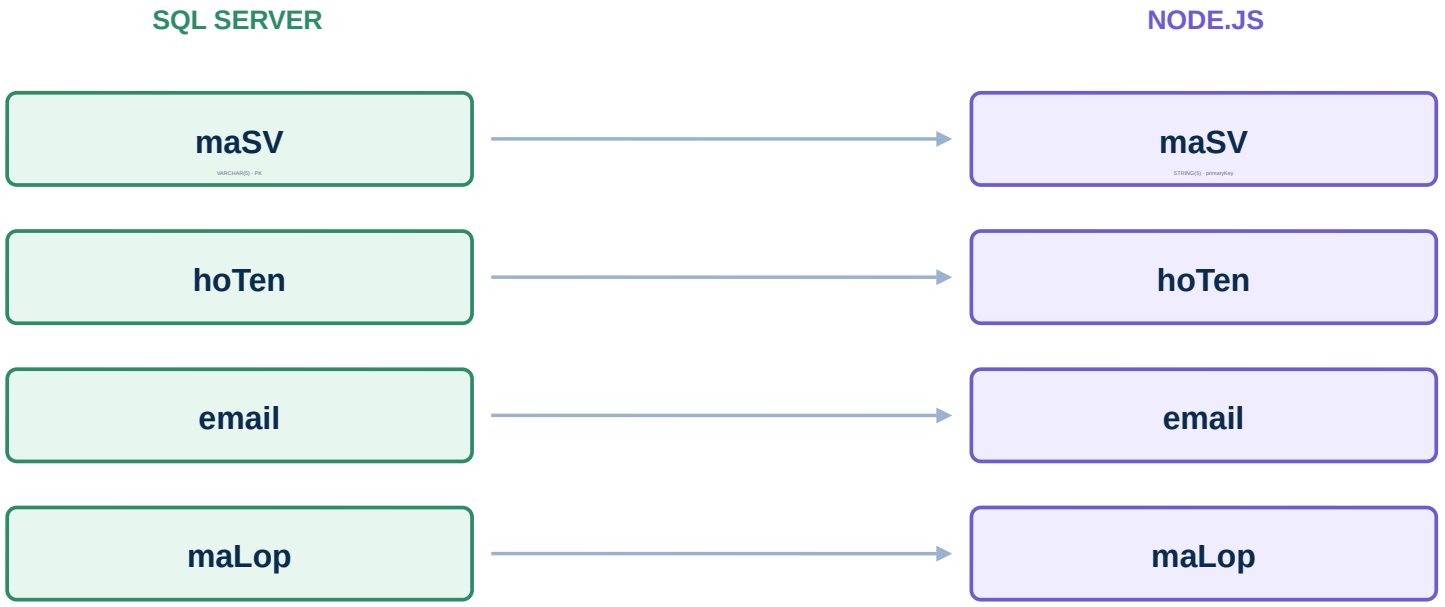
Yêu cầu AI chỉ tạo đúng file kết nối; không Model, không HTTP server, không hard-code secret.

ĐIỂM DỪNG Terminal xác thực được đúng QuanLySinhVienDB; code không sync/tạo bảng.

MODEL - CẦU NỐI GIỮA CODE VÀ BẢNG

Sau khi kết nối DB, code vẫn cần biết cấu trúc bảng SinhVien

LÝ THUYẾT - TABLE ↔ SEQUELIZE MODEL



Sequelize Model.init(...) ánh xạ field, kiểu dữ liệu, tableName, schema, timestamps.

DTO
Dữ liệu đi vào/đi ra API.

Model
Ánh xạ dữ liệu lưu trong bảng.

THỰC HÀNH

Tạo src/models/SinhVien.ts

1. Class SinhVien extends Model; Model.init với 4 trường.
2. tableName="SinhVien", schema="dbo", timestamps=false.
3. maSV primaryKey=true; không autoIncrement, không tự thêm id.
4. Giữ nguyên ISinhVienDTO cũ.

PROMPT GỢI Ý
Cung cấp schema thật cho AI; cấm AI tự thêm field ngoài Database.

ĐIỂM DỪNG Model khớp đúng schema SQL; DTO vẫn tồn tại độc lập.

SERVICE: THAY MẢNG RAM BẰNG MODEL

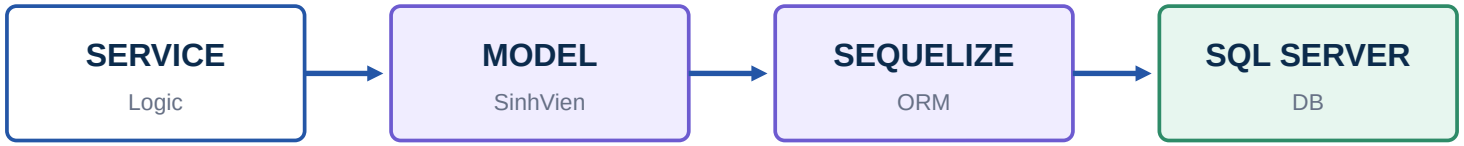
Đây là chỗ thay đổi quan trọng nhất - Route gần như vẫn giữ nguyên

LÝ THUYẾT - THAY NGUỒN DỮ LIỆU, KHÔNG THAY HỢP ĐỒNG API

TRƯỚC



SAU



findAll() = danh sách | findByPk() = theo mã | create() = thêm mới

Vì Database I/O là bất đồng bộ → các hàm Service trở thành async và trả Promise.

THỰC HÀNH

Chuyển sinhVien.service.ts sang DB

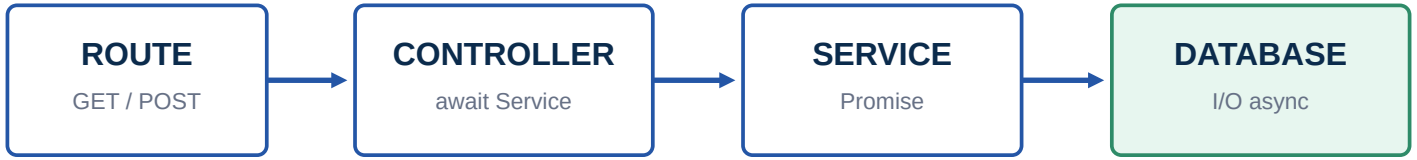
1. Xóa mảng sinhVienList khỏi đường đi chính.
2. layDanhSachSinhVien → findAll order maSV ASC.
3. timSinhVienTheoMa → findByPk; null → undefined.
4. themSinhVien → validate → create; xử lý trùng mã 409.

ĐIỂM DỪNG GET và POST cùng đọc/ghi SQL Server; không còn “POST ghi RAM, GET đọc DB”.

CONTROLLER PHẢI AWAIT SERVICE

Khi Service trở thành async, Controller phải chờ dữ liệu trước khi trả Response

LÝ THUYẾT - ASYNC/AWAIT TRONG LUỒNG HTTP




```
const data = await layDanhSachSinhVien();

res.status(200).json({ success: true, message, data });
```

200	201	400	404 / 409
GET thành công	POST tạo mới	Validation sai	Không có / trùng mã

THỰC HÀNH

Sửa Controller, giữ hợp đồng HTTP

1. Đổi 3 handler sang async/await.
2. Giữ GET /, GET /:maSV, POST / và response success/message/data.
3. Không đưa SQL/Model trực tiếp vào Controller.
4. try/catch → next(error); không gửi Response hai lần.

ĐIỂM DỪNG Build đạt; JSON không chứa Promise hoặc Sequelize internals.

KHỞI ĐỘNG SERVER ĐÚNG THỨ TỰ

Database phải sẵn sàng trước khi Express bắt đầu nhận Request

LÝ THUYẾT - STARTUP FLOW



KHÔNG LÀM **Không listen hai lần · Không sequelize.sync() · Không seed/reset dữ liệu mỗi lần server khởi động.**

Mục tiêu: nếu DB không kết nối được thì server không giả vờ "chạy bình thường".

THỰC HÀNH

Cập nhật server.ts

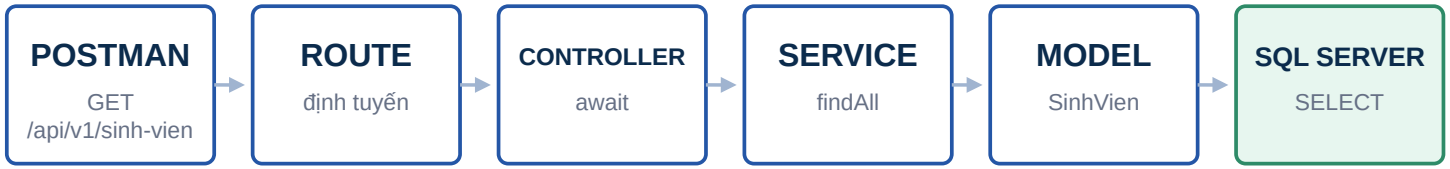
1. Import sequelize từ ./config/database.js.
2. Trong start(): await sequelize.authenticate() trước app.listen.
3. Giữ middleware, Route, 404 và error middleware đúng thứ tự.
4. Chạy pnpm build → check:db → dev; sau đó test pnpm start.

ĐIỂM DỪNG DB OK trước HTTP; / và /api/v1/sinh-vien vẫn hoạt động.

KIỂM CHỨNG DỮ LIỆU THẬT - GET + SSMS

HTTP 200 chưa đủ; phải đối chiếu Response với dữ liệu thật trong bảng

LÝ THUYẾT - ĐỌC DỮ LIỆU TỪ DATABASE



Postman Response

```
{
 "maSV":"SV001",
 "hoTen":"Minh Nhật",
 "email":"...",
 "maLop":"25CT114"
}
```

SSMS

```
SELECT maSV,hoTen,email,maLop
FROM dbo.SinhVien
ORDER BY maSV;
```

Hai nguồn phải khớp.

THỰC HÀNH

Dùng lại Collection Postman

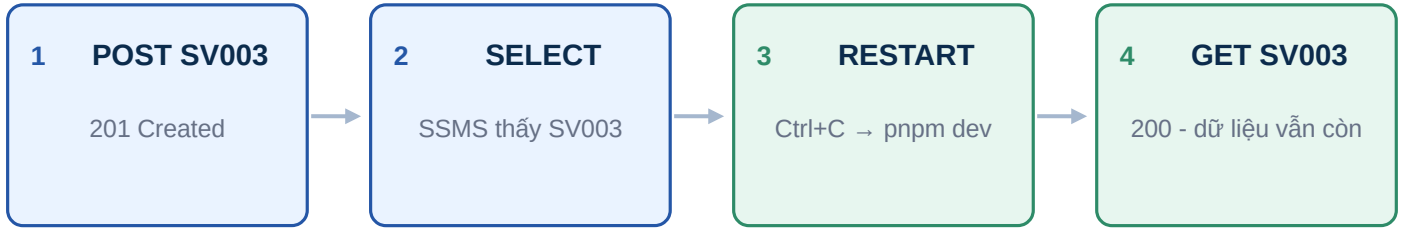
1. GET /, GET danh sách, GET SV001.
2. So sánh đủ 4 trường với SELECT trong SSMS.
3. Thêm Postman test: status 200, success=true, data là array.
4. Không chỉ nhìn “200 OK” rồi kết luận đã đúng.

ĐIỂM DỪNG Response Postman trùng dữ liệu trong dbo.SinhVien.

KIỂM CHỨNG LƯU BỀN VỮNG - POST → RESTART → GET

Đây là bằng chứng quan trọng nhất cho việc đã rời khỏi RAM

LÝ THUYẾT - PERSISTENCE



BODY MẪU

```
{"maSV":"SV003","hoTen":"Ngọc Mai","email":"ngocmai@example.com","maLop":"25CT114"}
```

POST lại cùng SV003 → 409 Conflict và chỉ có 1 bản ghi.

THỰC HÀNH

Bài test “SV003 không biến mất”

1. POST SV003 → kỳ vọng 201.
2. GET SV003 và SELECT trong SSMS → cùng dữ liệu.
3. Restart Node.js → GET SV003 lại.
4. POST trùng mã → 409; GET danh sách chỉ một SV003.

ĐIỂM DỪNG Restart không làm mất dữ liệu - chứng minh dữ liệu nằm ở SQL Server.

VALIDATION & LỖI SAU KHI CHUYỂN DATABASE

Chuyển nơi lưu dữ liệu không được làm hỏng quy tắc nghiệp vụ cũ

LÝ THUYẾT - STATUS CODE & DATA INTEGRITY

400	**Dữ liệu sai**	Thiếu hoTen, maSV sai format, email sai, vượt độ dài
404	**Không tìm thấy**	Mã sinh viên không có / URL không tồn tại
409	**Xung đột**	maSV đã tồn tại
500	**Lỗi hệ thống**	Lỗi DB chưa biết; không lộ SQL/secret

Nguyên tắc: Request bị từ chối phải không để lại bản ghi lỗi trong Database.

THỰC HÀNH

Chạy test ma trận lỗi

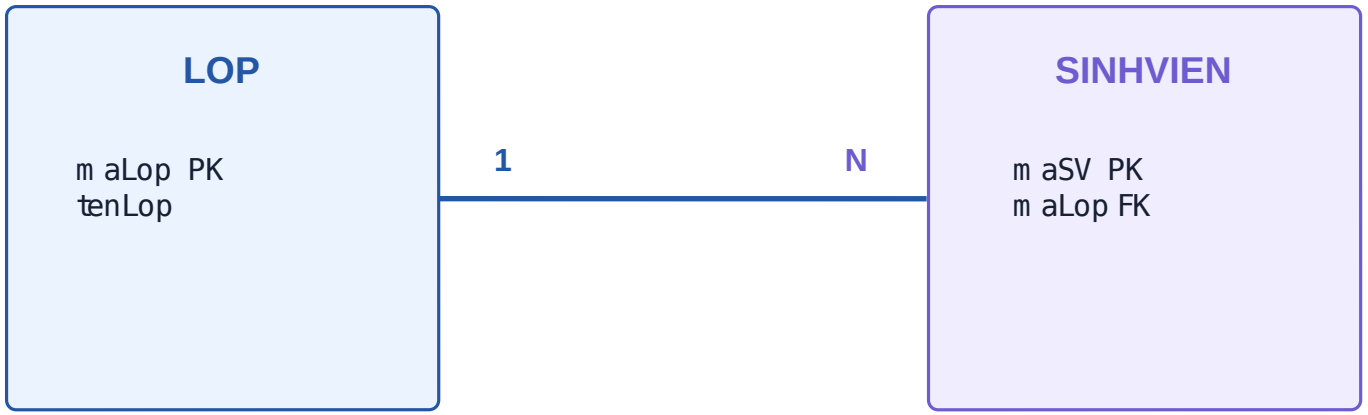
1. Thiếu/rỗng hoTen, maLop; maSV sai; email sai → 400.
2. hoTen >150, email >254, maLop >50 → 400; không cắt chuỗi.
3. JSON hỏng → 400 JSON dễ hiểu.
4. GET mã không có → 404; POST trùng → 409; lỗi DB khác → 500.

ĐIỂM DỪNG Sau mỗi ca lỗi, GET/SELECT chứng minh bản ghi sai không được lưu.

MỞ RỘNG: QUAN HỆ LỚP 1 - N SINH VIÊN

Chỉ làm sau khi bài chính đọc/ghi SQL Server đã chạy đúng

LÝ THUYẾT - ERD → FK → ORM ASSOCIATION



`Lop.hasMany(SinhVien)`

`SinhVien.belongsTo(Lop)`

Khi thêm FK, quy tắc nghiệp vụ mới xuất hiện: maLop gửi lên phải tồn tại trong bảng Lop.

THỰC HÀNH

Phần mở rộng sau bài chính

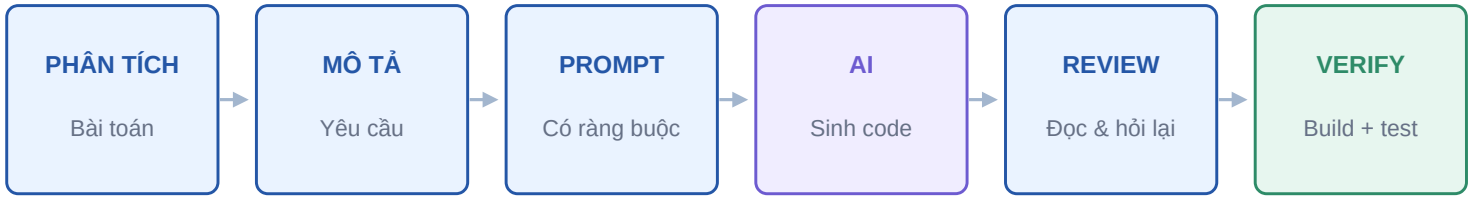
1. Tạo bảng Lop và thêm FK SinhVien.maLop → Lop.maLop.
2. Tạo Lop Model và khai báo hasMany / belongsTo.
3. Kiểm tra POST mã lớp không tồn tại → 400.
4. Không thay response chính; không tự thêm CRUD lớp.

ĐIỂM DỪNG Sinh viên nối được ERD → FK SQL → association trong ORM.

VIBE CODING CÓ KIỂM SOÁT

AI sinh code; người học quyết định yêu cầu, đọc code, kiểm tra và chứng minh

LÝ THUYẾT - QUY TRÌNH LÀM VIỆC VỚI AI



ROLE
AI đóng vai gì?

CONTEXT
Project đang có gì?

TASK
Cần làm file nào?

CONSTRAINT
Không được đổi gì?

OUTPUT
Muốn AI trả ra sao?

CODE REVIEW
“Dòng này làm gì? Nếu sai password thì đi đâu? Model map field nào?”

THỰC HÀNH

Nhật ký Prompt + Review

1. Mỗi bước chỉ yêu cầu AI sửa đúng phần cần thiết.
2. Không gửi .env/mật khẩu thật cho AI.
3. Sau code AI: build, chạy test và đọc file đã đổi.
4. Ghi lại prompt, kết quả, lỗi và điều chỉnh của bạn.

PROMPT GỢI Ý
Khi lỗi: gửi lệnh/request, kỳ vọng, lỗi nguyên văn, file liên quan; yêu cầu sửa nhỏ nhất.

ĐIỂM DỪNG Sinh viên giải thích được code AI viết, không chỉ “copy rồi chạy”.

KẾT THÚC BUỔI 4 - ĐẦU RA PHẢI ĐẠT

Chưa cần thuộc hết Sequelize; cần hiểu đúng luồng và chứng minh hệ thống dùng dữ liệu thật

KIẾN THỨC CẦN NẮM

SQL Server
Nơi lưu dữ liệu thật

ORM
Ánh xạ Object ↔ Table

Sequelize
ORM dùng trong project

Model
Mô tả bảng trong code

.env
Cấu hình môi trường

database.ts
Thiết lập kết nối

Service
Đọc/ghi qua Model

Postman
Kiểm chứng API

Persistence
Restart không mất dữ liệu

BUỔI SAU: RESTful API & CRUD - mở rộng GET / POST sang PUT / DELETE khi nền Database đã vững.

THỰC HÀNH

Hồ sơ nộp cuối buổi

1. Source + sql + package.json + pnpm-lock + .env.example; không nộp .env.
2. Collection Postman có test GET/POST/persistence/validation.
3. Ảnh bằng chứng: check:db, SELECT, POST 201, GET sau restart, 409, 400.
4. README ngắn: thứ tự SQL, lệnh chạy, tài khoản app, điều kiện test.

ĐIỂM DỪNG Có thể tự trả lời: DTO khác Model? vì sao await? PK chống trùng ra sao? vì sao POST/GET phải cùng nguồn?